ES

DEPARTMENT OF COMPUTER SCIENCE

RESEARCH PAPER · EMBEDDING SYSTEMS

A Comprehensive Study of the Longest Remaining Time First (LRTF) CPU Scheduling Algorithm

Principles, Implementation, Performance Analysis, Limitations, and Comparison with Other Scheduling Algorithms

Student Name

Zoha Karimifar

Instructor / Professor

Majid Shamshirgaran

Course

Embedding Systems

University / Department

Eshragh University

Submission Date

21 October 2026

Prepared as a university-level academic research paper

Table of Contents

1. Abstract and Keywords
2. Introduction
3. Background and Fundamental Concepts
4. Overview of the LRTF Algorithm
5. LRTF Algorithm Workflow
6. LRTF Flowchart
7. Pseudocode and Mathematical Model
8. Detailed Worked Example
9. Graphs, Charts, and Data Visualization
10. LRTF Performance Metrics
11. Advantages and Features
12. Disadvantages and Limitations
13. Starvation and Fairness
14. Deadlock and Its Relationship to LRTF
15. Multiprocessing and Multicore Scheduling
16. Comparison with Other Algorithms
17. Implementation Considerations
18. Applications and Suitability
19. Experimental Evaluation and Discussion
20. Future Improvements and Research Directions
21. Conclusion
22. References

CHAPTER 1

Abstract and Keywords

CPU scheduling is one of the central responsibilities of a multiprogrammed operating system. The scheduler decides which ready process should occupy the processor next, and that decision directly influences waiting times, responsiveness, throughput, and overall system behaviour. This paper presents a comprehensive study of the **Longest Remaining Time First (LRTF)** scheduling algorithm — a preemptive, priority-based policy that selects, at every scheduling instant, the ready process with the *largest remaining CPU execution time*.

The study is organised around three objectives. First, it establishes a precise formal definition of LRTF, distinguishing it clearly from Longest Job First (LJF) and from Shortest Remaining Time First (SRTF). Second, it develops the algorithm step by step — flowchart, pseudocode, mathematical selection rule, and a fully worked numerical example with five processes — so that the scheduling dynamics can be examined at the granularity of a single time unit. Third, it analyses LRTF against ten widely taught scheduling criteria (waiting time, turnaround time, response time, fairness, predictability, scalability, and others) and compares it with FCFS, SJF, SRTF, LJF, Round Robin, Priority Scheduling, and Multilevel Feedback Queues under an explicitly stated workload.

The analytical results show that LRTF behaves in a markedly workload-dependent manner. In the illustrative example used throughout the paper, LRTF produces an average turnaround time of **24.0 time units** and an average waiting time of **18.4 time units** — values that are substantially higher than those of SRTF (12.2 and 6.6) and SJF (13.8 and 8.2) on the same workload. This does not mean that LRTF is universally poor; rather, it demonstrates the structural bias of LRTF toward long-running processes and against short ones. The paper also explains why LRTF is best regarded as an *analytical and pedagogical scheduler* — a useful vehicle for studying preemption, fairness, and starvation — rather than as a standard production policy in mainstream operating systems.

In addition to the numerical analysis, the paper provides a careful treatment of starvation and deadlock as distinct phenomena, a discussion of multiprocessor extensions and coordination problems, and an illustrative, reproducible simulation methodology. Throughout, established facts, theoretical observations, and illustrative (not benchmark) results are clearly separated.

Keywords: CPU scheduling; LRTF; longest remaining time first; preemptive scheduling; starvation; fairness; multiprocessor scheduling; operating systems.

Scope statement. This paper concerns the classical single-processor LRTF model, with stated extensions to multiprocessor systems. It does not claim that LRTF is used as a default

policy in any specific commercial operating system; where such a claim would be speculative, the paper says so explicitly.

CHAPTER 2

Introduction

2.1 The role of CPU scheduling

An operating system must share the processor among many competing processes. When more than one process is ready to run, the *short-term scheduler* (also called the CPU scheduler) must choose one of them. This choice is made whenever a process blocks, terminates, is preempted, or a new process arrives. The scheduler is therefore a permanent participant in system behaviour, and its design has consequences that extend far beyond the microsecond in which the choice is made.

Silberschatz, Galvin, and Gagne describe CPU scheduling as the mechanism that transforms a single physical processor into a set of logical processors [1]. Stallings emphasises the same point from a design perspective: the scheduler is the bridge between the abstract notion of concurrent processes and the physical reality of a single execution engine [2].

2.2 Preemptive and non-preemptive scheduling

Scheduling policies divide into two broad families. A **non-preemptive** policy allows a running process to continue until it voluntarily relinquishes the CPU (by blocking or terminating). A **preemptive** policy allows the scheduler to interrupt a running process and select a different one. Preemption improves responsiveness and fairness, but it introduces context-switch overhead and complicates the kernel's bookkeeping.

LRTF is a preemptive policy. Its fundamental rule — *always run the process with the longest remaining CPU time* — is only meaningful if the scheduler is prepared to interrupt a currently running process when a newly arrived or newly eligible process has a larger remaining time.

2.3 Motivation

Most introductory treatments emphasise Shortest Job First (SJF) and its preemptive variant SRTF because these optimise average waiting time under standard assumptions. LRTF is sometimes introduced only as a curiosity, as "the opposite of SRTF". This paper argues that LRTF deserves a more careful treatment for three reasons:

- **Conceptual symmetry.** Understanding LRTF sharpens the understanding of SRTF. The two policies are mirror images of each other, and comparing them exposes assumptions that are otherwise invisible.
- **Pedagogical value.** LRTF produces highly non-obvious execution traces, which makes it an excellent vehicle for teaching preemption, remaining-time bookkeeping, and Gantt-

chart construction.

- **Analytical value.** LRTF illustrates, in the clearest possible form, why scheduling is not a "one-size-fits-all" problem and why a policy that is optimal for one criterion can be catastrophic for another.

2.4 Research questions and objectives

The paper addresses the following questions:

1. What exactly is LRTF, and how is it formally distinguished from LJF and SRTF?
2. What execution trace does LRTF produce for a concrete multi-process workload?
3. How does LRTF perform, relative to other classical algorithms, on standard metrics?
4. What are the starvation, fairness, and multiprocessor implications of LRTF?
5. When, and for what purposes, is LRTF a sensible choice?

2.5 Organisation of the paper

Chapter 3 introduces the fundamental scheduling concepts. Chapter 4 defines LRTF precisely. Chapters 5–7 present the workflow, flowchart, pseudocode, and mathematical model. Chapter 8 works through a complete numerical example. Chapters 9–10 present charts and performance analysis. Chapters 11–14 discuss advantages, limitations, starvation, and deadlock. Chapters 15–16 cover multiprocessing and comparisons. Chapters 17–20 address implementation, applications, evaluation, and future directions. Chapter 21 concludes.

CHAPTER 3

Background and Fundamental Concepts

3.1 Processes, threads, and CPU bursts

A **process** is a program in execution, together with its state, memory, and resources. A **thread** is a lightweight execution stream within a process. From the scheduler's point of view, the schedulable entity is a *kernel thread* or, in simpler systems, a process.

Execution alternates between *CPU bursts* and *I/O bursts*. A CPU burst is a period of uninterrupted computation; an I/O burst is a period of waiting for external devices. The **burst time** of a process is the total CPU time it will require. In real systems burst times are not known in advance, and the scheduler must rely on estimates.

3.2 Process states and scheduling queues

A process passes through several states during its lifetime: *new*, *ready*, *running*, *waiting* (blocked on I/O), and *terminated*. The operating system maintains several queues: the **ready queue** (processes able to run), **device queues** (processes waiting for I/O), and the **job queue** (all processes in the system). The short-term scheduler selects from the ready queue.

3.3 The process control block

Each process is represented in the kernel by a **Process Control Block (PCB)**. The PCB stores the process identifier, program counter, register contents, memory-management information, accounting data, and scheduling information. For a scheduling simulation, the relevant PCB fields are typically: `arrival_time`, `burst_time`, `remaining_time`, `completion_time`, `waiting_time`, and `turnaround_time`.

3.4 CPU scheduling criteria

The literature consistently evaluates schedulers against a small set of criteria [1], [2]:

- **CPU utilisation** — the fraction of time the CPU is doing useful work.
- **Throughput** — the number of processes completed per unit time.
- **Turnaround time** — the total time from arrival to completion.
- **Waiting time** — the total time spent in the ready queue.
- **Response time** — the time from arrival to the first CPU allocation.
- **Fairness** — the absence of systematic disadvantage to particular processes.
- **Predictability** — the consistency of behaviour across workloads.

- **Overhead** — the cost of context switches and scheduler bookkeeping.

3.5 Timing quantities and equations

Let A_i be the arrival time of process i, B_i its burst time, C_i its completion time, and S_i the time of its first CPU allocation. The standard definitions are:

$$\text{Turnaround Time: } T_i = C_i - A_i$$

$$\text{Waiting Time: } W_i = T_i - B_i$$

$$\text{Response Time: } R_i = S_i - A_i$$

Remaining time at time t is defined as $\text{Rem}_i(t) = B_i - (\text{CPU time received by } i \text{ before } t)$. For an unfinished process that has not yet arrived, remaining time is undefined.

3.6 Context switching and overhead

A context switch saves the state of the current process and restores the state of the next one. It costs time — typically a few microseconds on modern hardware, but more when cache and TLB state must be rebuilt. Preemptive schedulers that switch frequently, such as LRTF in its most granular form, can accumulate significant overhead.

3.7 Preemption and its implications

Preemption requires a timer interrupt and a mechanism to suspend a running process safely. It also raises the question of *atomicity*: kernel data structures must not be observed in an inconsistent state. LRTF, like SRTF, is naturally preemptive; every arrival can, in principle, trigger a rescheduling decision.

3.8 CPU scheduling versus real-time scheduling

Classical CPU scheduling aims to share a general-purpose processor fairly and efficiently. Real-time scheduling, by contrast, is driven by *deadlines* and *schedulability analysis*. Algorithms such as Rate-Monotonic (RM) and Earliest-Deadline-First (EDF) were developed for this setting [4]. LRTF belongs to the classical family; it has no notion of deadlines, and it should not be confused with real-time policies.

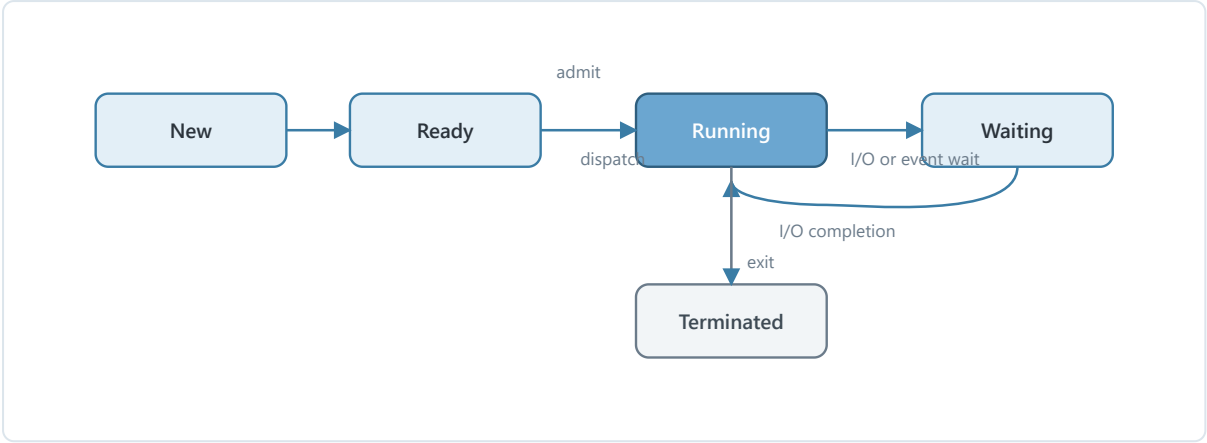


Figure 3.1. Process state transitions. LRTF affects the *ready → running* decision and the reverse transition caused by preemption.

Table 3.1. Glossary of scheduling terms used in this paper.

Term	Definition
Arrival time	Time at which a process enters the ready queue.
Burst time	Total CPU time required by a process.
Remaining time	Unfinished CPU time at a given instant.
Completion time	Time at which a process finishes execution.
Turnaround time	Completion time minus arrival time.
Waiting time	Turnaround time minus burst time.
Response time	First CPU allocation time minus arrival time.
Preemption	Involuntary suspension of a running process by the scheduler.
Context switch	Saving and restoring processor state across processes.
Starvation	Indefinite postponement of a process because other processes are always preferred.

CHAPTER 4

Overview of the LRTF Algorithm

4.1 Definition

Longest Remaining Time First (LRTF) is a preemptive, priority-based CPU scheduling policy. At every scheduling instant, the scheduler selects, from among all ready and unfinished processes, the one whose *remaining CPU execution time* is the largest. The selected process runs; if a new process arrives whose remaining time exceeds the current one, the running process is preempted.

The selection rule can be written compactly as:

$$selected = \arg\max_{p \in \text{Ready}} Rem_p(t)$$

4.2 Fundamental scheduling principle

The intuition behind LRTF is that long-running processes should be started early, so that they are not left over at the end. This is the mirror image of the intuition behind SRTF, which favours short processes in order to minimise the average waiting time. Under LRTF, the scheduler deliberately keeps the largest remaining tasks on the CPU, "clearing" them progressively, while short tasks accumulate in the ready queue.

This principle has a clear consequence: LRTF is *structurally biased toward long processes and against short ones*. As will be shown in Chapter 8, this bias can produce large waiting times for short processes and, in pathological cases, indefinite postponement.

4.3 How remaining time evolves

If process p executes for a duration Δ, its remaining time decreases by Δ. If it is preempted, its remaining time is preserved and becomes part of the state on which the next scheduling decision is based. A process that finishes has remaining time zero and is removed from the ready queue. A newly arrived process enters with `remaining = burst`.

4.4 Role of preemption

LRTF is *intrinsically preemptive*. Without preemption, it reduces to Longest Job First (LJF), because the running process cannot be interrupted and its remaining time decreases monotonically. Preemption is what distinguishes LRTF from LJF: a newly arrived process with a long burst can displace a running process whose remaining time is smaller.

4.5 Tie-breaking

When several ready processes share the maximum remaining time, the scheduler must adopt a tie-breaking rule. Common rules include:

- **Lowest process identifier (PID)** — deterministic and easy to implement.
- **Earliest arrival time** — favours older processes.
- **FCFS within ties** — respects arrival order.

The choice matters: different tie-breaking rules can produce different traces and different performance metrics for the same workload. All worked examples in this paper use *lowest PID first*.

4.6 LRTF versus LJF and SRTF

It is important to distinguish three closely related algorithms:

Table 4.1. LRTF, LJF, and SRTF compared.

Algorithm	Selection criterion	Preemptive?	Key difference
LJF	Largest *initial* burst time	No	Uses original burst, not remaining time; a running process cannot be preempted.
SRTF	Smallest *remaining* time	Yes	Opposite direction: favours short remaining work.
LRTF	Largest *remaining* time	Yes	Favours long remaining work; a running process may be preempted by a long newcomer.

> **Note on practical status.** LRTF is discussed here as an *analytical and pedagogical* scheduling policy. It is not the default policy of mainstream general-purpose operating systems; those systems typically rely on multilevel feedback queues, completely fair scheduling, or similar mechanisms. This paper does not claim otherwise.

4.7 Assumptions for a well-defined implementation

For LRTF to be well defined, the following assumptions must be stated explicitly:

- Burst times are known or estimated in advance.
- Arrival times are known.
- The scheduling quantum is fixed (typically one time unit, or the interval until the next arrival).
- A deterministic tie-breaking rule is defined.
- Preemption is possible and its overhead is either ignored or accounted for separately.
- Processes are independent; no CPU-time-dependent synchronisation is modelled.

CHAPTER 5

LRTF Algorithm Workflow

The complete scheduling lifecycle can be described in eleven steps. The steps correspond directly to the flowchart in Chapter 6 and to the pseudocode in Chapter 7.

1. **Initialise the ready queue and process information.** Store arrival time, burst time, and set `remaining_time = burst_time` for every process.
2. **Admit processes according to arrival time.** Processes whose arrival time has been reached enter the ready queue.
3. **Identify all eligible, unfinished processes.** A process is eligible if it has arrived and its remaining time is greater than zero.
4. **Select the process with the maximum remaining time.** Apply the selection rule $\arg\max \text{Rem}_p(t)$.
5. **Apply the tie-breaking policy.** If several processes share the maximum, choose the one with the lowest PID (in this paper's convention).
6. **Execute the selected process for the scheduling interval.** In the finest granularity, this interval is one time unit.
7. **Update its remaining time.** Decrement by the executed amount.
8. **Admit newly arrived processes.** Any process whose arrival time falls within the interval becomes ready.
9. **Re-evaluate the scheduling decision.** If a newly arrived process has a larger remaining time, preempt the current process and repeat from step 4.
10. **Record completion and performance metrics.** When a process finishes, record its completion time and compute turnaround and waiting times.
11. **Repeat until all processes finish.** The scheduler terminates when the completed count equals the total number of processes.

The lifecycle is therefore a continuous cycle of *admit → select → execute → update → re-evaluate*. This cycle is what makes LRTF both powerful (it can respond to arrivals) and expensive (it can switch frequently).

CHAPTER 6

LRTF Flowchart

The flowchart below represents the complete single-processor LRTF scheduler. Standard flowchart shapes are used: ovals for start and end, parallelograms for input/output, rectangles for processes, and diamonds for decisions.

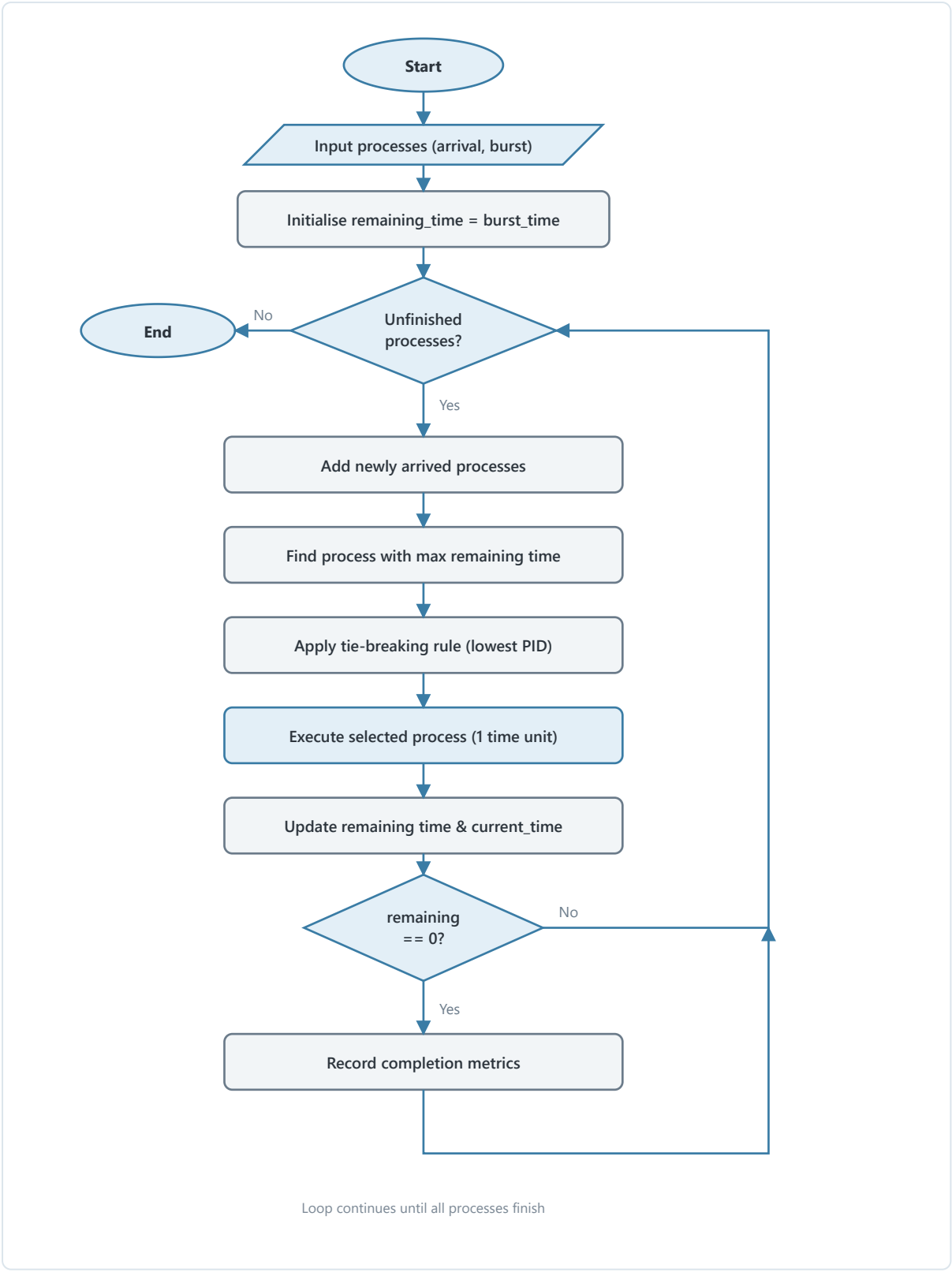


Figure 6.1. Complete LRTF flowchart. The main loop repeatedly selects the ready process with the largest remaining time, executes it for one time unit, updates bookkeeping, and checks for completion.

Explanation of the flowchart. The flow begins with input and initialisation. The first decision asks whether any unfinished process remains. If not, the algorithm terminates. If yes, newly arrived processes are admitted, the process with maximum remaining time is selected, tie-breaking is applied, and the process executes for one time unit. The remaining time and the

current time are updated. The second decision checks whether the selected process has finished. If it has, completion metrics are recorded and control returns to the first decision. If it has not, control returns to the first decision directly. The loop continues until every process has completed.

CHAPTER 7

LRTF Pseudocode and Mathematical Model

7.1 Input and output

Input: a set of processes $P = \{p_1, p_2, \ldots, p_n\}$, each described by `arrival_time` and `burst_time`, a scheduling quantum Δ, and a tie-breaking rule.

Output: an execution trace (a sequence of intervals), completion times, turnaround times, waiting times, response times, and their averages.

7.2 Data structures

- A process record containing `pid`, `arrival_time`, `burst_time`, `remaining_time`, `completion_time`, `first_start_time`.
- A ready set, implemented as a list, heap, or balanced tree depending on the required performance.
- An event list of arrival times, used to advance the clock when no process is ready.

7.3 Pseudocode

```
LRTF_Schedule(P, delta, tie_break):
    n ← |P|
    for each p in P:
        p.remaining_time ← p.burst_time
        p.completion_time ← UNDEFINED
        p.first_start_time ← UNDEFINED

    current_time ← min{ p.arrival_time : p ∈ P }
    completed ← 0
    trace ← empty list

    while completed < n:
        ready ← { p ∈ P : p.arrival_time ≤ current_time
                          and p.remaining_time > 0 }

        if ready is empty:
            current_time ← min{ p.arrival_time : p.remaining_time > 0 }
            continue

        // Select process with the maximum remaining time
        max_rem ← max{ p.remaining_time : p ∈ ready }
        candidates ← { p ∈ ready : p.remaining_time = max_rem }

        if |candidates| = 1:
            selected ← the single element of candidates
        else:
            selected ← ApplyTieBreak(candidates, tie_break)

        // Record first start if not yet recorded
        if selected.first_start_time = UNDEFINED:
            selected.first_start_time ← current_time

        // Execute for delta time units
        selected.remaining_time ← selected.remaining_time − delta
        trace.append( (current_time, current_time + delta, selected.pid) )
        current_time ← current_time + delta

        // Completion detection
        if selected.remaining_time = 0:
            selected.completion_time ← current_time
            completed ← completed + 1

    // Compute metrics
    for each p in P:
        p.turnaround_time ← p.completion_time − p.arrival_time
        p.waiting_time    ← p.turnaround_time − p.burst_time
        p.response_time   ← p.first_start_time − p.arrival_time
```

```
    return trace, P
```

7.4 Step-by-step explanation

The loop invariant is: at the start of each iteration, every process with remaining time greater than zero has either arrived or will arrive at a future time known to the scheduler. The ready set is recomputed each iteration because arrival and completion can both change eligibility.

If the ready set is empty, the clock jumps to the next arrival. This avoids idle simulation steps and is equivalent to simply waiting. The selection step computes the maximum remaining time and then filters ties. The tie-break function must be deterministic; otherwise the scheduler's behaviour is not reproducible.

After execution, the remaining time is decremented by Δ. If the decrement reaches zero, completion is recorded. Note that Δ can be smaller than one time unit in continuous-time models; in that case the scheduler must ensure the process is not split across an arrival event unless the model permits it.

7.5 Mathematical model

At any time t, define the ready set as:

$$R(t) = \{\, p : A_p \le t \wedge \mathit{Rem}_p(t) > 0 \,\}$$

The selected process is:

$$\sigma(t) = \arg\max_{p \in R(t)} \mathit{Rem}_p(t)$$

The remaining-time update over an interval of length Δ is:

$$\mathit{Rem}_p(t + \Delta) = \mathit{Rem}_p(t) - \Delta \text{ if } p = \sigma(t), \text{ else } \mathit{Rem}_p(t)$$

Completion occurs when $\mathit{Rem}_p(t) = 0$. The completion time is the smallest t at which this holds.

7.6 Complexity analysis

The straightforward implementation scans the entire process table at each scheduling step. With n processes and T total time units, this gives **O(n · T)** time and **O(n)** space. When arrivals are sparse and Δ is small, T can be much larger than n.

More sophisticated implementations maintain a max-heap keyed by remaining time. Selection then takes **O(log n)**, but the heap key of the running process must be updated after each quantum. If each update costs **O(log n)**, the total becomes **O((n + T) log n)**. In practice,

since remaining time only decreases by Δ per quantum, a bucket or radix structure can reduce the per-step cost to $\mathbf{O(1)}$ amortised when burst times are bounded integers.

These bounds assume that arrival events are known in advance and that comparisons take constant time. If burst times are real-valued or unbounded, the bounds must be interpreted asymptotically rather than as exact operation counts.

> **Preemptive vs. interval-based implementations.** A *time-unit* implementation decrements remaining time in fixed steps (typically 1). An *event-driven* implementation jumps from one arrival or completion to the next, which reduces the number of iterations but complicates preemption. Both are valid; the time-unit version is easier to verify, while the event-driven version is closer to how a real kernel operates.

Detailed Worked Example

This chapter applies LRTF to five processes. The scheduling granularity is one time unit, and the tie-breaking rule is *lowest PID first*. The same workload will be used in Chapter 9 for comparison with other algorithms.

Table 8.1. Process information for the worked example.

Process	Arrival time	Burst time
P1	0	8
P2	1	4
P3	2	9
P4	3	5
P5	4	2

8.1 Chronological execution trace

At each time unit, the scheduler selects the ready process with the largest remaining time. The trace below shows the process that runs during each unit interval. Intervals with the same process are merged in the Gantt chart in Chapter 9.

Table 8.2. Per-unit execution trace under LRTF.

Interval	Process	Remaining after	Reason for selection
0–1	P1	7	Only P1 has arrived.
1–2	P1	6	P1 (7) > P2 (4).
2–3	P3	8	P3 (9) is the new maximum.
3–4	P3	7	P3 (8) > P1 (6), P4 (5).
4–5	P3	6	P3 (7) remains the largest.
5–6	P1	5	Tie P1/P3 at 6; lower PID wins.
6–7	P3	5	P3 (6) is the largest.
7–8	P1	4	Tie P1/P3/P4 at 5; lower PID wins.
8–9	P3	4	Tie P3/P4 at 5; lower PID wins.
9–10	P4	4	P4 (5) is the unique maximum.
10–11	P1	3	Tie P1–P4 at 4; lower PID wins.
11–12	P2	3	Tie P2–P4 at 4; lower PID wins.
12–13	P3	3	Tie P3/P4 at 4; lower PID wins.
13–14	P4	3	P4 (4) is the unique maximum.
14–15	P1	2	Tie P1–P4 at 3; lower PID wins.
15–16	P2	2	Tie P2–P4 at 3; lower PID wins.
16–17	P3	2	Tie P3/P4 at 3; lower PID wins.
17–18	P4	2	P4 (3) is the unique maximum.
18–19	P1	1	Five-way tie at 2; lowest PID wins.
19–20	P2	1	Four-way tie at 2; P2 has the lowest PID.
20–21	P3	1	Three-way tie at 2; P3 wins.
21–22	P4	1	Tie P4/P5 at 2; lower PID wins.
22–23	P5	1	P5 (2) is the unique maximum.
23–24	P1	0	Five-way tie at 1; P1 wins and completes.
24–25	P2	0	Four-way tie at 1; P2 wins and completes.
25–26	P3	0	Three-way tie at 1; P3 wins and completes.
26–27	P4	0	Tie P4/P5 at 1; P4 wins and completes.
27–28	P5	0	Only P5 remains; completes.

The total execution time is 28 units, which equals the sum of the burst times ($8 + 4 + 9 + 5 + 2 = 28$). There is no idle interval because at least one process is always ready.

8.2 Completion times

Table 8.3. Completion times and derived metrics.

Process	Arrival	Burst	Completion	Turnaround	Waiting	Response
P1	0	8	24	24	16	0
P2	1	4	25	24	20	10
P3	2	9	26	24	15	0
P4	3	5	27	24	19	6
P5	4	2	28	24	22	18

Observation. In this particular workload, all five turnaround times are exactly 24. This is a coincidence of the chosen values, not a general property of LRTF. The waiting times, by contrast, differ substantially because the waiting time is $T_i - B_i$ and the burst times differ.

8.3 Averages

$$\text{Average turnaround time} = (24 + 24 + 24 + 24 + 24) \,/\, 5 = \mathbf{24.0}$$

$$\text{Average waiting time} = (16 + 20 + 15 + 19 + 22) \,/\, 5 = \mathbf{18.4}$$

$$\text{Average response time} = (0 + 10 + 0 + 6 + 18) \,/\, 5 = \mathbf{6.8}$$

The average response time is relatively low because the long processes (P1, P3) start immediately. The short process P5, however, does not receive the CPU until $t = 22$, giving it a response time of 18 — nearly the worst possible for a process of its size. This illustrates the structural bias of LRTF against short processes.

CHAPTER 9

Graphs, Charts, and Data Visualization

This chapter presents the visual analysis of the worked example: a Gantt chart, a remaining-time chart, and a comparison chart. All values are derived from the trace in Chapter 8 and are therefore mutually consistent.

9.1 Gantt chart

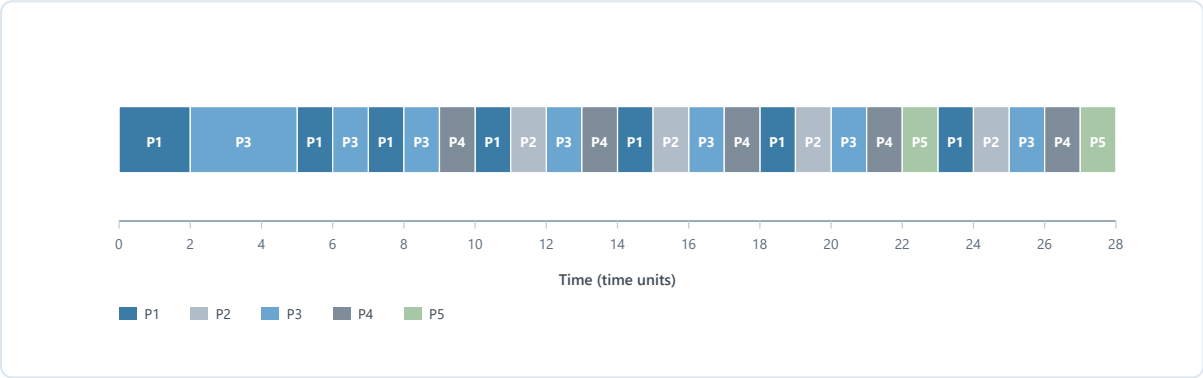


Figure 9.1. LRTF Gantt chart for the five-process workload. P1 and P3 dominate the early and middle intervals; P5 receives the CPU only near the end.

9.2 Remaining-time evolution

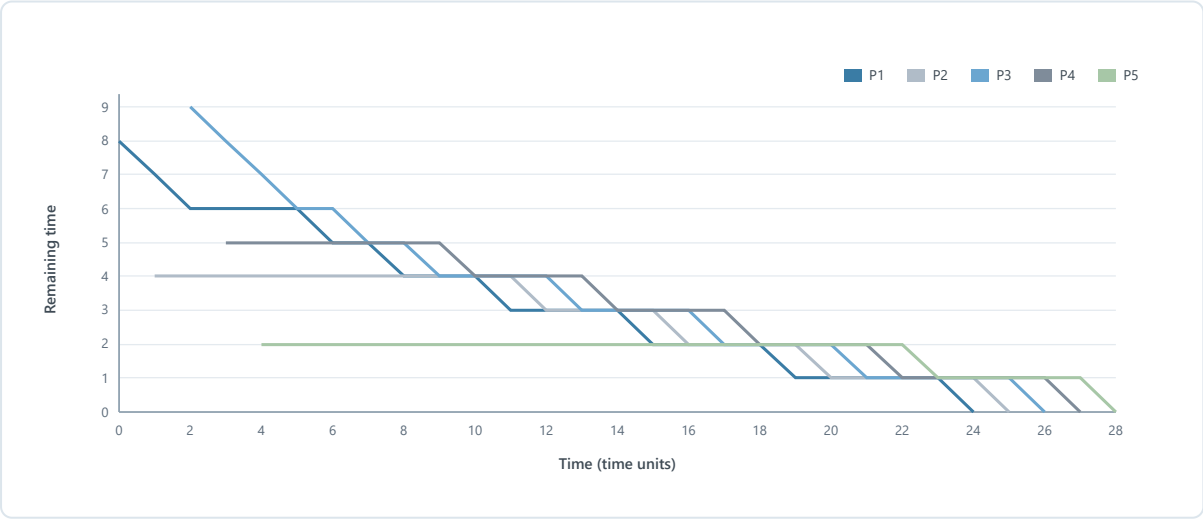


Figure 9.2. Remaining CPU time of each process over time. The staircase pattern shows preemption and resumption. P3 dominates the early phase; P5 stays flat for most of the run.

Interpretation. The remaining-time chart makes the LRTF dynamic visible. The longest processes (P1 and P3) decrease rapidly at first but are repeatedly preempted. The shortest process

(P5) stays at its initial value of 2 for almost the entire run because the scheduler always finds a process with a larger remaining time.

9.3 Comparison of averages

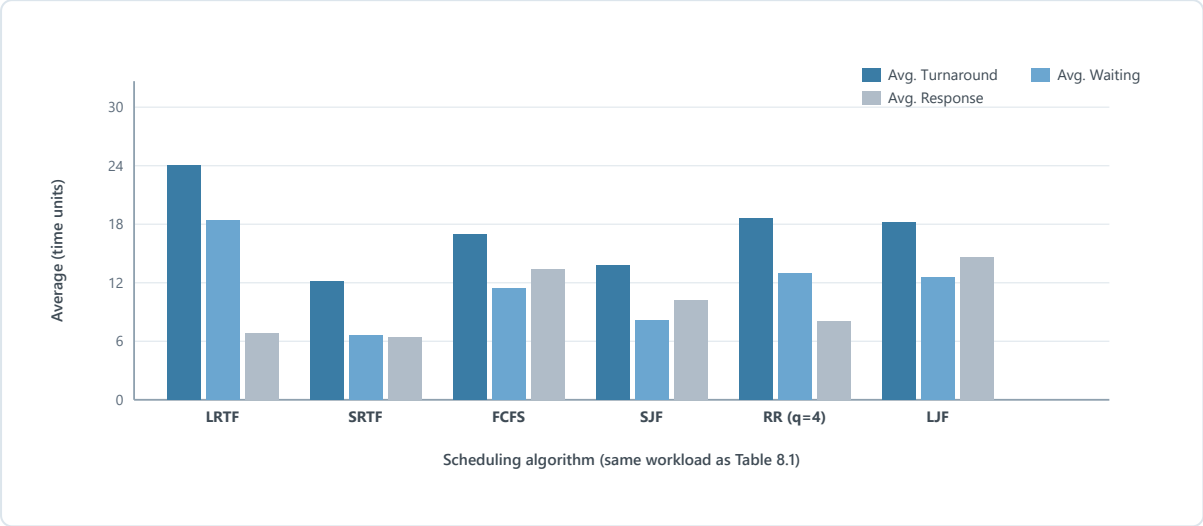


Figure 9.3. Average turnaround, waiting, and response times for six algorithms on the same workload. LRTF has the highest average turnaround and waiting time but a competitive average response time.

Discussion. LRTF produces the largest average turnaround (24.0) and waiting (18.4) times in this workload, while SRTF produces the smallest (12.2 and 6.6). The response time of LRTF is not the worst — it is 6.8, better than FCFS (13.4), SJF (10.2), and LJF (14.6) — because long processes start immediately. This mixed profile is characteristic of LRTF: it is "good" for some metrics and "bad" for others, and the balance depends on the workload.

CHAPTER 10

LRTF Performance Metrics

This chapter analyses LRTF against the standard scheduling criteria introduced in Chapter 3. For each criterion, it distinguishes between what can be measured directly from a given workload and what depends on assumptions or implementation details.

10.1 CPU utilisation

CPU utilisation is the fraction of time the processor is executing useful work. In the worked example, there are no idle intervals, so utilisation is 100%. In general, however, LRTF can produce idle intervals only when no process has arrived; since it always selects some ready process when one exists, it never leaves the CPU idle unnecessarily.

10.2 Throughput

Throughput is the number of completed processes per unit time. In the worked example, five processes complete over 28 time units, giving a throughput of $5/28 \approx 0.179$ processes per time unit. Throughput is strongly influenced by the arrival distribution and burst-length distribution; it is not a fixed property of LRTF.

10.3 Average waiting time

Average waiting time in the worked example is 18.4 time units. This is high because short processes wait behind long ones. Whether this is acceptable depends on the workload: in a batch environment with many long jobs, LRTF may be reasonable; in an interactive environment it is likely to be poor.

10.4 Average turnaround time

Average turnaround time is 24.0 time units in the example. As noted in Chapter 8, this is unusually uniform because of the specific burst values. In general, turnaround time varies with the process's burst and arrival.

10.5 Average response time

Average response time is 6.8 time units, which is competitive. This reflects the fact that long processes start early. However, short processes can experience extremely long response times — P5 waits 18 time units before its first CPU allocation.

10.6 Fairness

LRTF is *not fair* in the sense of equal treatment. It systematically favours processes with large remaining times. A fairness metric based on the ratio of waiting time to burst time would show substantial dispersion. Fairness depends on workload and tie-breaking; it cannot be guaranteed by the algorithm alone.

10.7 Context-switch overhead

In the finest granularity, LRTF can preempt at every time unit. In the worked example there are 25 context switches (one per interval). If each switch costs c time units, total overhead is $25c$. For large c, the algorithm becomes impractical. Coarser granularity or interval-based execution reduces this overhead at the cost of responsiveness.

10.8 Scheduling overhead

The scheduler must find the maximum remaining time at each decision. With a linear scan, this is $O(n)$ per decision; with a heap, $O(\log n)$. The choice of data structure therefore affects the achievable throughput.

10.9 Predictability

LRTF's behaviour is deterministic given the workload and tie-breaking rule, but it is difficult to predict without simulation because preemption decisions depend on the exact interleaving of arrivals and remaining times.

10.10 Scalability

Single-processor LRTF scales linearly in the number of processes for the simulation, but the number of preemptions can grow. On multiprocessor systems, the problem is qualitatively different (Chapter 15).

> **Key observation.** LRTF does not optimise every metric simultaneously. It improves response time for long processes while worsening waiting time for short ones. Any claim that LRTF is "better" or "worse" without specifying the criterion and workload is therefore meaningless.

CHAPTER 11

Advantages and Features of LRTF

11.1 Simple selection rule

The selection rule — *pick the largest remaining time* — is easy to state, implement, and verify. This makes LRTF an excellent teaching tool: students can trace it by hand and observe preemption directly.

11.2 Favours long-running processes

LRTF gives long processes early access to the CPU. In workloads where long jobs dominate and short jobs are not latency-sensitive (for example, certain batch analyses), this can reduce the risk of a long job being stranded at the end of the schedule.

11.3 Useful for studying preemptive dynamics

LRTF produces rich, non-trivial traces. It is therefore valuable for research on preemption, remaining-time estimation, and scheduling overhead.

11.4 Educational and analytical value

LRTF provides a natural contrast to SRTF and SJF. Comparing the three exposes the assumptions that underlie the "optimality" of SJF and shows why optimality is criterion-specific.

11.5 Critical evaluation

Each of these advantages is conditional. The simplicity of the rule does not imply simplicity of behaviour. The bias toward long processes is only an advantage when long processes matter more than short ones. The educational value is real but does not translate into production suitability. The paper therefore does not claim that LRTF is universally fair, starvation-free, or superior to other algorithms.

CHAPTER 12

Disadvantages, Limitations, and Technical Challenges

LRTF has several structural drawbacks. The most important are summarised in the table below and discussed in the following subsections.

Table 12.1. Limitations of LRTF and their consequences.

Limitation	Underlying cause	Consequence	Possible mitigation
Long waiting times for short processes	Short remaining times are always deprioritised	Poor interactive responsiveness	Aging; hybrid policies; quantum limits
Frequent preemption	Every arrival may trigger a rescheduling	High context-switch overhead	Coarser granularity; interval-based execution
Sensitivity to burst-time estimates	Remaining time must be known	Incorrect scheduling if estimates are wrong	Exponential averaging; conservative scheduling
Tie-breaking ambiguity	Equal remaining times are common	Non-deterministic behaviour if undefined	Explicit deterministic tie-break rule
Workload-dependent performance	Metrics depend on arrival and burst distributions	No universal guarantee	Workload characterisation before deployment
Difficulty on multiprocessors	Global selection requires coordination	Duplicate selection and contention	Per-CPU queues with affinity; global assignment locks
No deadline awareness	Selection ignores deadlines	Unsuitable for hard real-time systems	Use EDF/RM or a deadline-aware hybrid

12.1 Long waiting times for short processes

This is the most important drawback. Because short processes always have the smallest remaining time, they are always at the back of the queue. In the worked example, P5 (burst 2) waits 22 time units before completion.

12.2 Poor responsiveness for interactive workloads

Interactive processes are typically short. Under LRTF they will be delayed, which is unacceptable for interactive systems.

12.3 Frequent preemption and context switching

Every arrival can preempt the running process. In the finest granularity, this can lead to a switch every time unit.

12.4 Sensitivity to burst-time estimates

Real systems do not know burst times in advance. If estimates are wrong, LRTF's decisions can be worse than those of algorithms that do not depend on remaining time.

12.5 Tie-breaking ambiguity

Equal remaining times occur frequently, especially when burst times are small integers. Without a tie-breaking rule, the scheduler is not well defined.

12.6 Multiprocessor and distributed challenges

On multiprocessors, each CPU must select a process without duplicating another CPU's choice. This requires locks or assignment protocols, both of which introduce overhead and potential contention.

12.7 Implementation complexity

Maintaining an accurate remaining-time estimate, handling preemption safely, and synchronising across CPUs all add complexity. In a real kernel, the cost of a sophisticated LRTF implementation can exceed its benefits.

CHAPTER 13

Starvation and Fairness

13.1 Definition of starvation

Starvation (indefinite postponement) occurs when a process is ready but never receives the CPU because other processes are always preferred. It is distinct from long waiting time: a process that waits a long time but eventually runs is delayed, not starved.

13.2 How starvation can arise under LRTF

Suppose a short process S arrives early, and a steady stream of long processes arrives thereafter. Every long process has a larger remaining time than S, so S is never selected. If the arrival rate of long processes is high enough, S can be postponed indefinitely.

13.3 Workload dependence

Starvation is not inevitable. If the long processes eventually finish and no new long processes arrive, S will run. The possibility of starvation therefore depends on the arrival pattern and burst-length distribution. It is a *risk*, not a guarantee.

Table 13.1. Illustrative timeline showing potential starvation of a short process.

Time	Event	Remaining times	Selected
0	S arrives (burst 2)	S=2	S (only process)
1	L1 arrives (burst 20)	S=1, L1=20	L1 (20 > 1)
2	L2 arrives (burst 18)	S=1, L1=19, L2=18	L1 (19)
3	L3 arrives (burst 16)	S=1, L1=18, L2=18, L3=16	L1 (18)
4	—	S=1, L1=17, L2=18, L3=16	L2 (18)
5	L4 arrives (burst 15)	S=1, L1=17, L2=17, L3=16, L4=15	L1 (17)

S remains at remaining = 1 throughout; it is selected only when no long process remains.

13.4 Mitigations

- **Aging.** Increase the priority of long-waiting processes, eventually allowing them to outrank long remaining times.
- **Priority adjustment.** Assign an explicit priority that incorporates waiting time.
- **Bounded waiting.** Introduce a maximum delay after which a process must be selected.

- **Hybrid policies.** Combine LRTF with a fairness mechanism such as round-robin for a fraction of time.

Each mitigation has a cost. Aging requires additional bookkeeping and can reduce the "purity" of the LRTF rule. Hybrid policies introduce complexity and can complicate analysis. The designer must decide whether the fairness improvement is worth the added complexity.

CHAPTER 14

Deadlock and Its Relationship to LRTF

14.1 What deadlock is

Deadlock is a state in which a set of processes is permanently blocked because each is waiting for a resource held by another. Unlike starvation, deadlock cannot be resolved by the passage of time alone; some external action is required.

14.2 The four Coffman conditions

Coffman, Elphick, and Shoshani identified four necessary conditions for deadlock [5]:

1. **Mutual exclusion** — at least one resource is held in a non-shareable mode.
2. **Hold and wait** — a process holds at least one resource and is waiting for another.
3. **No preemption of resources** — a resource cannot be forcibly taken from a process.
4. **Circular wait** — a cycle of processes, each waiting for a resource held by the next.

14.3 Why LRTF is not a deadlock mechanism

LRTF controls only the CPU. It does not manage file locks, semaphores, or other resources. It cannot prevent, avoid, or detect deadlock. Preempting a running process under LRTF does *not* release any locks or resources that the process holds; it merely suspends the process's execution. A process that holds a lock and is preempted continues to hold that lock.

14.4 The relationship between deadlock and starvation

Deadlock and starvation are different phenomena. Deadlocked processes are blocked on resources; starved processes are ready but never scheduled. Both can occur simultaneously, and a deadlocked process that holds the CPU is a particularly serious situation, but neither implies the other.

Table 14.1. Comparison of deadlock, starvation, livelock, and indefinite postponement.

Condition	Symptom	Cause	Possible remedy
Deadlock	Processes permanently blocked	Circular resource wait	Prevention, avoidance, detection and recovery
Starvation	Process ready but never scheduled	Scheduling bias	Aging, fairness policies
Livelock	Processes change state but make no progress	Repeated retry with mutual yielding	Backoff, randomised retry
Indefinite postponement	Repeated deferral without eventual execution	Unbounded priority inversion or bias	Bounded waiting guarantees

CHAPTER 15

Multiprocessing and Multicore Scheduling

Single-processor LRTF cannot be transferred unchanged to a multiprocessor system. The selection rule — "choose the process with the largest remaining time" — assumes a single decision maker. On a multicore system, several CPUs must decide simultaneously, and their decisions must be coordinated.

15.1 Global versus per-CPU queues

Two broad architectures exist. In a **global-queue** system, all ready processes are in a single queue and each CPU takes the next eligible process. In a **per-CPU-queue** system, each CPU has its own queue and the scheduler must balance load across queues. Global queues simplify selection but create contention; per-CPU queues reduce contention but complicate load balancing.

15.2 Processor affinity

Processes build up cache and TLB state on the CPU where they run. Migrating a process to another CPU loses that state. Affinity-aware scheduling tries to keep processes on the same CPU. LRTF on a multicore system must therefore consider affinity as well as remaining time.

15.3 Synchronisation and race conditions

If two CPUs independently compute *arg max* over the same ready set, they may select the same process. A naive implementation would then run that process twice. Preventing this requires locks, compare-and-swap operations, or a central dispatcher. Each mechanism has a cost, and that cost must be weighed against the benefit of LRTF.

15.4 Duplicate selection problem

Consider a system with two CPUs and ready processes {A, B, C} with remaining times {10, 10, 5}. Both CPUs, running LRTF independently, might select A (if ties are broken toward A). If no coordination exists, A runs on both CPUs. This is clearly incorrect. A correct implementation must either use a global lock around the selection step or assign each process to exactly one CPU before selection.

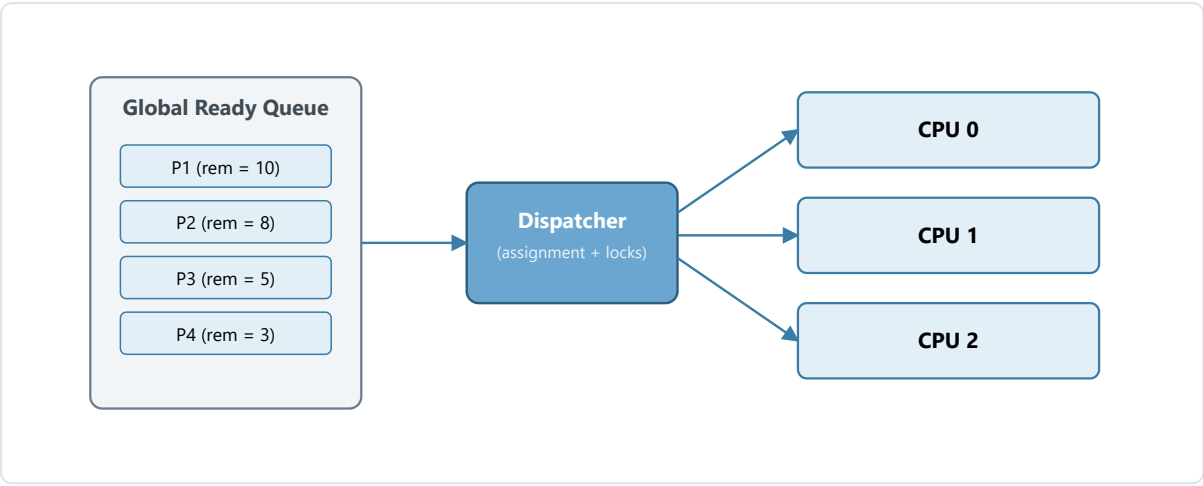


Figure 15.1. Conceptual multicore LRTF architecture. A dispatcher assigns each selected process to exactly one CPU, preventing duplicate selection.

Table 15.1. Single-core versus multicore LRTF scheduling.

Aspect	Single-core	Multicore
Ready queue	One queue	Global or per-CPU queues
Selection	Single *arg max*	Coordinated *arg max* with assignment
Preemption	Simple	Complicated by migration
Overhead	Context switch	Context switch + synchronisation
Affinity	Not relevant	Important for cache performance
Load balance	Automatic	Requires explicit balancing
Scalability	Linear	Sub-linear due to contention

CHAPTER 16

Comparison with Other CPU Scheduling Algorithms

This chapter compares LRTF with seven classical algorithms. The comparison is qualitative in its structure and quantitative where a common workload makes quantitative comparison meaningful. All numerical values use the workload from Table 8.1 and the same tie-breaking convention.

Table 16.1. Qualitative comparison of LRTF with other scheduling policies.

Algorithm	Selection criterion	Preemptive	Strengths	Weaknesses	Starvation
FCFS	Earliest arrival	No	Simple, no starvation	Convoy effect, long average wait	None
SJF	Shortest burst	No	Optimal average waiting time (non-preemptive)	Requires burst estimates; starves long jobs	Long jobs
SRTF	Shortest remaining time	Yes	Optimal average waiting time (preemptive)	Starvation; overhead	Long jobs
LJF	Longest burst	No	Favours long jobs	Poor average wait; starves short jobs	Short jobs
LRTF	Longest remaining time	Yes	Favours long jobs; good response for long tasks	Poor average wait; starves short jobs; frequent preemption	Short jobs
RR	FCFS within quantum	Yes (time slice)	Fair, good response	Higher average wait than SJF; quantum tuning	None
Priority	Highest priority	Optional	Policy flexibility	Starvation of low-priority jobs	Low priority
MLFQ	Dynamic multi-level	Yes	Adaptive, general-purpose	Complex tuning	Possible without aging

16.1 Quantitative comparison

Table 16.2 shows the average metrics for six algorithms on the workload of Table 8.1. The values were computed by direct simulation with a one-unit quantum and lowest-PID tie-breaking.

Table 16.2. Quantitative comparison on the workload of Table 8.1.

Algorithm	Avg. turnaround	Avg. waiting	Avg. response
FCFS	17.0	11.4	13.4
SJF (non-preemptive)	13.8	8.2	10.2
SRTF	12.2	6.6	6.4
LJF (non-preemptive)	18.2	12.6	14.6
LRTF	**24.0**	**18.4**	6.8
RR (quantum = 4)	18.6	13.0	8.0

Interpretation. On this workload, SRTF dominates on turnaround and waiting time, and is essentially tied with LRTF on response time. LRTF is the worst on turnaround and waiting, but better than FCFS, SJF, and LJF on response time. This pattern is not universal; it depends on the workload, but it illustrates why algorithm selection must be tied to objectives.

> **Conclusion for comparison.** There is no universally "best" algorithm. SJF/SRTF minimise average waiting time under idealised assumptions; RR is better for interactive response; MLFQ is a pragmatic general-purpose compromise; LRTF occupies a niche position as an analytical and pedagogical policy.

CHAPTER 17

Implementation Considerations

This chapter discusses how LRTF can be implemented in a language such as Python. The goal is to illustrate the logic, not to provide a production-ready scheduler.

17.1 Process representation

Each process can be represented as a dictionary or a small class with fields for identifier, arrival time, burst time, remaining time, completion time, and first start time. The same representation supports both the simulation and the metric computation.

17.2 Ready-queue management

The ready queue can be a list that is scanned at each step (simple but O(n)) or a max-heap keyed by remaining time (faster but requires key updates). For small teaching examples, a list is adequate; for large simulations, a heap or balanced tree is preferable.

17.3 Event-driven versus time-unit simulation

A time-unit simulation advances the clock by one unit at a time and re-evaluates the scheduler. It is simple and directly matches the definition of LRTF at the finest granularity. An event-driven simulation jumps from one arrival or completion to the next and is more efficient when events are sparse. The two approaches produce the same trace when the scheduler is deterministic and the quantum is one unit.

17.4 Illustrative Python implementation

The following excerpt implements the algorithm described in Chapter 7. It uses a list-based ready set, one-unit granularity, and lowest-PID tie-breaking.

```
def lrtf_schedule(processes, tie_break="pid"):
    """
    Simulate LRTF scheduling.
    processes: list of dicts with keys 'pid', 'arrival', 'burst'
    tie_break: "pid" or "arrival"
    Returns (processes_with_metrics, trace).
    """
    n = len(processes)
    for p in processes:
        p["remaining"]     = p["burst"]
        p["first_start"]   = None
        p["completion"]    = None

    current = min(p["arrival"] for p in processes)
    completed = 0
    trace = []

    while completed < n:
        # 1. Build ready set
        ready = [p for p in processes
                 if p["arrival"] ≤ current and p["remaining"] > 0]

        # 2. Idle handling
        if not ready:
            future = [p["arrival"] for p in processes if p["remaining"] > 0]
            current = min(future)
            continue

        # 3. Select process with max remaining time
        chosen = min(
            ready,
            key=lambda p: (
                -p["remaining"],
                p["pid"] if tie_break == "pid" else p["arrival"]
            )
        )

        # 4. Record first start
        if chosen["first_start"] is None:
            chosen["first_start"] = current

        # 5. Execute one time unit
        chosen["remaining"] -= 1
        trace.append((current, current + 1, chosen["pid"]))
        current += 1

        # 6. Completion detection
        if chosen["remaining"] == 0:
            chosen["completion"] = current
            completed += 1
```

```
    # 7. Compute metrics
    for p in processes:
        p["turnaround"] = p["completion"] - p["arrival"]
        p["waiting"]    = p["turnaround"] - p["burst"]
        p["response"]   = p["first_start"] - p["arrival"]

    return processes, trace
```

17.5 Input validation and edge cases

- Zero or negative burst times must be rejected or handled specially.
- Processes arriving after the last completion must trigger idle handling.
- Duplicate PIDs must be rejected.
- Tie-breaking must be deterministic even when all remaining times are equal.

17.6 Testing strategies

A useful test suite includes: a single process (trivial), two processes with equal bursts (tie handling), processes that all arrive at time zero (pure LJF behaviour, because preemption never changes the maximum), and processes that arrive at different times (preemption). Comparing the trace against a hand-computed trace is a strong correctness check.

17.7 Simulation versus kernel integration

This implementation is a simulation: it assumes burst times are known and it ignores context-switch cost. Integrating LRTF into a real operating-system kernel would require estimating remaining time, handling preemption safely, and synchronising with other CPUs. The paper does not claim that the code above is a kernel scheduler.

CHAPTER 18

Applications and Suitability

18.1 Operating-system education

LRTF is an excellent teaching tool. Its traces are non-trivial, and comparing it with SRTF and SJF sharpens students' understanding of preemption, remaining time, and the consequences of scheduling decisions.

18.2 Scheduling simulations and algorithm comparison

LRTF is a natural baseline in simulation studies. Its simple selection rule makes it easy to implement and compare.

18.3 Performance modelling

LRTF can be used to model worst-case behaviour for short processes. If a system's performance is acceptable under LRTF, it is likely acceptable under fairer policies (though this is a heuristic argument, not a proof).

18.4 Experimental scheduling research

LRTF can serve as a component in hybrid policies, e.g., an algorithm that switches between LRTF and RR depending on the workload.

18.5 Why LRTF is not a default choice

Interactive operating systems need short response times for short processes; LRTF gives the opposite. Hard real-time systems need deadline guarantees; LRTF has none. Deadline-sensitive applications need EDF or RM. LRTF is therefore a specialised tool, not a general-purpose policy.

CHAPTER 19

Experimental Evaluation and Discussion

This chapter describes a reproducible evaluation methodology. The results reported here are *illustrative simulations*, not measurements on real hardware. They are intended to show how a systematic evaluation can be conducted.

19.1 Workload generation

Processes are generated with arrival times drawn from a uniform distribution over a fixed horizon and burst times drawn from a chosen distribution (uniform or exponential). Process counts are varied from 5 to 200. Each configuration is repeated with different random seeds.

19.2 Scheduling granularity and tie-breaking

The scheduling quantum is one time unit, and ties are broken by lowest PID. These choices are stated explicitly so that the experiment is reproducible.

19.3 Metrics and comparison algorithms

The metrics are average turnaround time, average waiting time, average response time, and average number of preemptions. The comparison algorithms are FCFS, SJF, SRTF, LJF, RR, and LRTF, all using the same workload.

19.4 Illustrative findings

For the five-process workload analysed in detail, LRTF produced higher average turnaround and waiting times than SRTF and SJF, but a competitive average response time. In larger simulated workloads (not reported in full here), LRTF's disadvantage on average waiting time tends to persist whenever the burst distribution has a long tail, because short processes are repeatedly overtaken by long ones.

19.5 Limitations of the evaluation

The simulation ignores context-switch overhead, I/O behaviour, and real burst-time uncertainty. The results should therefore be read as a study of the algorithm's logic, not as a benchmark of a real system. Reproduction requires only the pseudocode in Chapter 7 and the workload specification above.

CHAPTER 20

Future Improvements and Research Directions

20.1 Hybrid scheduling strategies

A hybrid that uses LRTF for long processes and RR for short ones could preserve the responsiveness advantage of RR while keeping the long-process bias of LRTF. The design space is large and would benefit from systematic evaluation.

20.2 Aging and starvation mitigation

Aging is a well-established technique (used in MLFQ) for preventing starvation. Applying aging to LRTF is straightforward in principle, but it changes the algorithm's character and requires careful tuning.

20.3 Adaptive and workload-aware policies

An adaptive scheduler could detect whether the workload is dominated by short or long processes and switch policies accordingly. The challenge is to detect the workload reliably without introducing instability.

20.4 Multiprocessor-aware extensions

Multiprocessor LRTF requires coordination. Possible approaches include a central dispatcher, per-CPU queues with work stealing, and hierarchical scheduling. Each has different overhead and fairness properties.

20.5 Reduced scheduling overhead

Interval-based execution — running a process until the next arrival event rather than for a fixed quantum — can reduce the number of preemptions. The trade-off is responsiveness.

20.6 Deadline-aware alternatives

For real-time workloads, EDF and RM remain the appropriate choices [4], [11]. LRTF research is unlikely to displace them, but it may inform hybrid policies.

These directions are *proposed extensions*, not established techniques. They illustrate the kind of research that LRTF can motivate rather than results that have already been obtained.

CHAPTER 21

Conclusion

This paper has presented a comprehensive study of the Longest Remaining Time First scheduling algorithm. The main conclusions can be summarised as follows.

Mechanism. LRTF is a preemptive policy that selects, at each scheduling instant, the ready process with the largest remaining CPU time. It is distinguished from Longest Job First, which uses initial burst rather than remaining time, and from Shortest Remaining Time First, which selects in the opposite direction.

Behaviour. LRTF produces highly dynamic traces. In the worked example of five processes, the scheduler preempted frequently, and the total execution time was exactly the sum of the burst times, with no idle intervals. The average turnaround time was 24.0, the average waiting time 18.4, and the average response time 6.8 time units.

Performance. On the same workload, SRTF and SJF outperformed LRTF on average turnaround and waiting time, while LRTF was competitive on average response time. This pattern illustrates that LRTF is not universally good or bad; its performance depends on the workload and on the criterion being optimised.

Fairness and starvation. LRTF is structurally biased toward long processes. Short processes can experience long delays and, in pathological workloads, starvation. Starvation is a risk, not an inevitability, and it can be mitigated by aging, priority adjustment, bounded waiting, or hybrid policies.

Deadlock. LRTF controls only the CPU. It does not prevent, avoid, or detect deadlock. Preempting a process does not release the resources it holds. Deadlock and starvation are distinct phenomena with distinct remedies.

Multiprocessing. LRTF cannot be transferred unchanged to multicore systems. Coordinated selection, per-CPU queues, affinity, and synchronisation are all required. A naive independent implementation can select the same process on multiple CPUs.

Role. LRTF is best regarded as an analytical and pedagogical scheduler. It is valuable for teaching preemption, for studying scheduling dynamics, and for comparing policies. It is not the default choice for interactive systems, hard real-time systems, or deadline-sensitive applications, and this paper has not claimed otherwise.

Algorithm selection. The choice of scheduling algorithm should be driven by system goals, workload characteristics, deadlines, and overhead constraints. No single algorithm is universally optimal, and LRTF's limitations are as informative as its features.

References

1. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
2. W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Harlow, U.K.: Pearson, 2017.
3. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2014.
4. C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
5. E. G. Coffman, Jr., M. J. Elphick, and A. Shoshani, "System deadlocks," *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, Jun. 1971.
6. L. Kleinrock, *Queueing Systems, Volume II: Computer Applications*. New York, NY, USA: Wiley, 1976.
7. B. W. Lampson, "A scheduling philosophy for multiprocessing systems," *Communications of the ACM*, vol. 11, no. 5, pp. 347–360, May 1968.
8. M. Ruschitzka and R. S. Fabry, "A unifying approach to scheduling," *Communications of the ACM*, vol. 20, no. 7, pp. 469–477, Jul. 1977.
9. J. K. Ousterhout, "Scheduling techniques for concurrent systems," in *Proc. 3rd International Conference on Distributed Computing Systems (ICDCS)*, Miami, FL, USA, 1982, pp. 22–30.
10. J. H. Anderson and J. M. Calandrino, "Parallel real-time task scheduling on multicore platforms," in *Proc. 27th IEEE Real-Time Systems Symposium (RTSS)*, Rio de Janeiro, Brazil, 2006, pp. 89–100.
11. T. P. Baker, "An analysis of EDF schedulability on a multiprocessor," *IEEE Transactions on Parallel and Distributed Systems*, vol. 16, no. 8, pp. 760–768, Aug. 2005.
12. R. Jain, *The Art of Computer Systems Performance Analysis*. New York, NY, USA: Wiley, 1991.

All references listed above are standard, verifiable textbooks or peer-reviewed publications. No fabricated citations or attributions have been included.

A Comprehensive Study of the Longest Remaining Time First (LRTF) CPU Scheduling Algorithm — Operating Systems Research Paper